

# JavaScript-and-Vue — compiled path

Compiled from `/Users/darkonikolic/Documents/learning/paths/JavaScript-and-Vue`. Canonical sources stay in place.

## Phase — 01-javascript-fundamentals

Directory: `01-javascript-fundamentals/`

Source: `01-javascript-fundamentals/01-scope-runtime-scope-closures-and-async-model.md`

## Unit 1 — Scope: JavaScript fundamentals (why the runtime, not syntax bingo)

Mindset shift: from “syntax tricks” toward **executable mental model**.

### Learning outcomes

- **Scopes**: global, function, and block scopes; lexical environment; visibility across nested functions.
- `var`, `let`, `const` trade-offs; temporal dead-zone intuition for interviews.
- **Hoisting**: declarations vs bindings; declarations vs expressions; predict output before running.
- **Closures**: inner function retains outer bindings; encapsulation (“private”) without classes; misuse (leaks).
- **Prototypes**: lookup chain ( `[[Prototype]]` ); classical patterns vs ES classes as sugar; inheritance vs delegation.
- **Promises**: states; chaining; error propagation; migrating callback thinking.
- `async` / `await` as control-flow sugar over promises; pairing with `try/catch`.
- **Event loop sketch**: call stack, tasks (macrotasks), **microtasks** (promise jobs, `queueMicrotask`).
- **ES modules**: default vs named exports; folder layout ( `api/`, `utils/`, `modules/` ) before any framework.

Practice spine throughout this area (no Vue): `frontend-lab/` — plain `.js`, modules, deliberate exercises.

Source: `01-javascript-fundamentals/02-labs-counter-api-chain-and-mini-shop-plain-js.md`

## Unit 2 — Labs: drills + integration ( `frontend-lab/` )

### Micro-drills

- `counter.js`: block vs function scope; explain what each nested block “sees.”
- **Hoisting puzzle**: contrast `sayHello()` with declaration vs `hello()` via `const` /expression; predict then run.
- **Closure toolkit**: counters, memoisation-style cache, guarding mutable state behind a narrow API.
- **Prototype sketch**: lightweight `User` → `Admin` chain; articulate where lookups stop.
- **Promise chain**: at least three steps with `.then` / `.catch`; surface rejected branch.
- **Async flow**: sequential fake **users** → **orders** → **payment** APIs with `async` / `await` and explicit error path.
- **Event-loop prediction**: mixes of `console.log`, `setTimeout`, `Promise.resolve()` — write order on paper before execution.

### Integration ( `mini-shop/` )

Wire **modules**, **closures**, **async**, and **minimal state** inside cart / fake API flows. Document **why** an ordering or data boundary exists, not only what API you called.

Interview checklist from this phase: closures, prototype chain basics, promises & `async`, microtasks vs macrotasks, hoisting distinctions, `let` / `const` / `var`, module boundaries.

## Phase — 02-browser-runtime-and-apis

---

Directory: `02-browser-runtime-and-apis/`

---

Source: `02-browser-runtime-and-apis/01-scope-dom-rendering-network-and-storage.md`

## Unit 1 — Scope: browser as a runtime (not “HTML display”)

Mindset shift: from “I call DOM APIs” toward **rendering + network + storage + lifecycle** as a system.

### Learning outcomes

---

- **DOM as live tree**: query vs create vs mutate; cost of aggressive reflow/repaint.
- **Rendering pipeline** (conceptual): DOM + CSSOM → render tree → layout → paint → composite; when reflow vs repaint hurts.
- **Events**: listeners, bubbling vs capture, `stopPropagation` / `preventDefault` trade-offs.
- `fetch`: browser-side request lifecycle; loading / error UX hooks (even before Vue).
- **HTTP cache** in the browser: `Cache-Control`, validators (`ETag`, `Last-Modified`); hard vs normal refresh behaviour.
- `localStorage` **vs** `sessionStorage`: durability, tab scope, size and serialisation limits.
- **Memory**: detached DOM, forgotten listeners, growing caches — how leaks show up in DevTools.
- **Batching DOM work**: `DocumentFragment`, structural patterns to avoid 1k sequential hot-path updates.
- **DevTools**: Network, Performance, Memory — interpret slow requests, huge payloads, cache misses.

Practice spine: `browser-lab/` — still **no Vue**; plain JS against the real browser runtime. **Next** technical layer: `03-*` TypeScript discipline before framework work in `04-*`.

---

Source: `02-browser-runtime-and-apis/02-labs-todo-dom-fetch-mini-shop-browser.md`

## Unit 2 — Labs (`browser-lab/` / `mini-shop/`)

### Exercises

---

- **Todo DOM**: add/remove tasks with explicit event wiring; deliberate stress test (`appendChild` in a tight loop) to feel layout cost.
- **Rendering lab**: tweak width/font-size; contrast batched DOM updates vs naive per-item mutation.
- **Nested buttons**: bubbling vs capturing; document when you’d stop propagation.
- **Fake e-commerce HTTP**: parallel `fetch` for users/products/orders + explicit loading/error states.
- **Cache lab**: inspect Network panel for validators and cache behaviour; articulate hard refresh vs warm cache.
- **Persistence**: cart survives reload via `localStorage`; justify storage choice vs `sessionStorage`.
- **Leak hunt**: detach nodes but keep listeners; observe retention; fixed version with teardown.

### Integration (`mini-shop/`)

---

Reuse DOM + `fetch` + storage + render narrative. Attach a short doc: **render flow vs network flow vs where state lives**.

Interview focus: DOM vs repaint/reflow, event propagation, fetch semantics, caching headers, storage choice, leak patterns.

---

## Phase — 03-typescript-professional-systems-for-vue-and-http

---

Directory: 03-typescript-professional-systems-for-vue-and-http/

---

Source: 03-typescript-professional-systems-for-vue-and-http/01-scope-type-system-generics-unions-narrowing-and-composition.md

## Unit 1 — Scope: TypeScript at professional depth (before Vue + HTTP hardening)

You already own JavaScript execution semantics ( 01-\* ) and the browser runtime ( 02-\* ). This block upgrades you to the **type discipline** expected in senior Vue + API codebases: composable abstractions, safe narrowing, generics that survive refactors, and mental models sharp enough that `lang="ts"` is an asset—not a tarpit of `any`.

---

### Architectural outcomes

---

#### 1. Structural typing vs nominal habits

TypeScript compares **shape**, not nominal class names—except for intersections with private fields / branded types hacks. Interview articulation:

- Widening literal `const x = 'ok'` to `string` when reassigned across boundaries.
- **Excess property checking** only on fresh object literals—not on aliased bindings.
- **Readonly** differences vs mutable arrays/objects in inference.

Deliverable intuition: explain *why* two interfaces with identical fields unify in assignment even if “conceptually different business entities” unless you deliberately **brand**.

#### 2. Compiler flags & production posture

Operate with **strict-family** semantics in mind ( `strict`, `strictNullChecks`, `noUncheckedIndexedAccess` trade-offs, `exactOptionalPropertyTypes` footguns optional). Understand:

- `undefined` in optional props vs omitted keys when exact optional is on.
- **Non-null assertions** ( `!` ) as debt you must justify—not a default linter escape.

#### 3. Generics beyond syntax sugar

Professionals weaponise generics because **API boundaries move**:

- Constraints: `T extends Identifiable`, `T extends keyof SomeMap`, recursive constraints for tree-shaped JSON.
- **Multiple type parameters** with sensible defaults ( `TData = unknown` ).
- Inference sites: helper functions preserving unions vs accidentally widening.
- Variance intuition (arrays are invariant in mutation positions; `Readonly<T>` loosens covariance stories for callbacks—enough depth to avoid bogus “contravariant” hand-waving).

#### 4. Union types & discriminated unions

- Narrowing primitives: `string | number` with `typeof`.
- **kind discriminants**: exhaustiveness checks via `never` reducer or `switch (x.kind)` with `assertNever`.
- **Optional chaining + nullish coalescing** integrated with `strictNullChecks` mental model.

#### 5. Composition patterns (not “OOP theatre”)

- Intersection types `A & B` vs union `A | B` for layering DTOs.
- Mapped types teaser: `Pick`, `Omit`, `Partial`, `Required`, `Record`—implemented understanding, not only imports from `utility-types`.
- **Composition helpers**: small pure functions lifted through generics instead of fragile inheritance hierarchies.

## 6. Narrowing ergonomics ( `as const` , `satisfies` , predicates )

- `as const` for literal widening prevention + tuple preservation.
- `satisfies` **operator**: keep inference while asserting compatible shape ( `satisfies SomeConfig` ).
- **User-defined type guards** returning `value is Foo` justified by runtime validation (paired with `06-*` when HTTP JSON arrives).

## 7. Module graph & ambient types

Know when `.d.ts` augments globals vs when `import type` keeps emitted JS clean ( `verbatimModuleSyntax` awareness ).  
Respect **triple-slash** rarity—prefer modular imports.

## 8. Error modelling as typed data ( `Result` ergonomics pattern )

Professionals unify failure paths:

- Lightweight `Ok | Err` union with discriminant vs throwing across async boundaries—trade-offs with Vue/Pinia ergonomics discussed in `06-*` .
- Narrowing failures without `catch (e: any)` .

---

## SSR & hydration — conceptual backbone (engineering interview depth)

Separate **SSR** (initial HTML snapshot produced on a server) from **CSR-only** SPAs.

- **Why DB-backed dashboards** often mix SSR: faster first meaningful paint, SEO for product surfaces, consistent auth gating at the edge.
- **Hydration**: client runtime reattaches event listeners + reuses DOM; **mismatch** when server HTML  $\neq$  client virtual tree (common causes: random IDs, locale/timezone drift, `Date.now()` in render path, feature flags diverging per host).
- **Partial hydration / islands** as complexity trade-off—name the idea even if you do not implement a framework.

Link forward: production cache + security around tokens during SSR discussed in `09-*` .

---

## Accessibility (a11y) — minimum professional bar

You are not becoming a WCAG auditor here; you **own** the engineering basics that stop “pretty but unusable” data UIs:

- **Semantic landmarks** ( `main` , `nav` , headings hierarchy ) for screen-reader wayfinding.
- **Keyboard operability**: focus order, `tabindex` discipline, escape / enter patterns in dialogs.
- **Focus management** on SPA route changes (announce page context when reasonable).
- **Colour contrast & motion** awareness at spec level (where to look, not every SC number).
- Tooling: **Accessibility tree** in DevTools + one automated pass (axe / Lighthouse a11y) interpreted critically (false negatives/positives).

Forward link: component tests can assert **roles + labels** in `08-*` .

---

## Vue 3 + TypeScript — what you must preload

Before `04-*` :

- `<script setup lang="ts">` inference for `defineProps` / `defineEmits` with **generic components** patterns.
- `PropType` vs direct generic `defineProps<{ id: string }>()` ergonomics.
- **Ref / reactive** generic parameters; unwrapping in templates vs script (where inference stops).
- **Event payload typing**—no silent `any` on `$emit` .
- `withDefaults` for optional props + TypeScript interplay.

You will still practise raw components in `04-*` ; this block front-loads the **type muscle** so you do not fight the compiler while learning reactivity.

---

## Practice spine

---

Maintain `ts-professional-lab/` : small packages or Vite library apps (no full design system) where you can compile with strict settings and iterate fast.

Do **not** embed full Vue here beyond micro-SFC smoke tests when the lab demands—`04-*` owns component architecture depth.

---

**Source:** `03-typescript-professional-systems-for-vue-and-http/02-labs-ts-professional-lab-exercises.md`

## Unit 2 — Labs: `ts-professional-lab/` (strict, generic, narrow, compose)

Work in a **strict** TypeScript project (Vite `library` template or `tsc --noEmit + tsx` runner). Each exercise ends with a **one-paragraph explanation**: what would have broken without the type, and what remains **unsound** at runtime.

---

### Lab A — Generics that survive refactors

---

Implement:

```
function mapById<T extends { id: string }>(  
  rows: readonly T[],  
  selected: Iterable<string>,  
) : Map<string, T>;
```

- Accept **immutable** arrays; reject accidental widening of `selected` strings.
- Add overload or generic default so **callers infer** `T` from `rows`.

**Stretch:** second generic index signature safe access with `Record<string, T>` vs `Map` trade-off narrative.

---

### Lab B — Discriminated union + exhaustiveness

---

Model async API states:

```
type Remote<T> =  
  | { status: 'idle' }  
  | { status: 'loading' }  
  | { status: 'success'; data: T }  
  | { status: 'error'; code: number; retryAfter?: number };
```

Write `function describe<T>(state: Remote<T>): string`:

- Exhaustive switch with `assertNever` **helper** trapping future variants.
  - Unit-test that adding `{ status: 'stale'; data: T }` breaks compile until handled.
- 

### Lab C — User-defined narrowing from JSON (`unknown` ingestion)

---

Given `unknown` payload from `fetch`:

1. Implement **type guards** `isProduct`, `isProductList`.
2. Pair with `zod` **or** `valibot` (either is fine)—compare ergonomic errors vs handwritten guards.
3. Document **difference** between compile-time certainty and runtime validation.

(This is the hinge into `06-*` where HTTP clients centralise.)

---

### Lab D — Composition > inheritance

---

Compose pricing rules using **pure functions + intersections**:

```
type Money = { amount: number; currency: 'EUR' | 'USD' };
type Priced = { price: Money };
type DiscountRule = { kind: 'percent'; value: number } | { kind: 'fixed'; value: Money };
```

Build `applyDiscount(product: Priced & { rules: DiscountRule[] }): Money` **without classes**. Explain where **closed vs open extension** hurts you.

---

## Lab E — `satisfies` + `as const` configuration

---

Maintain a `routes` **table** keyed by literal path strings with metadata (`requiresAuth`). Use `as const satisfies Readonly<{...}>` to:

- Preserve literal keys for autocomplete.
- Forbid drifting optional flags silently.

Reflect: why `Record<string, ...>` often **destroys inference** compared to `satisfies`.

---

## Lab F — Branded IDs (nominal-lite)

---

Implement **branded primitives**:

```
type UserId = string & { readonly __brand: 'UserId' };
```

Functions `parseUserId(raw: string): UserId | null` vs unsafe cast—document phishing scenario where two stringly IDs collide conceptually.

---

## Lab G — Generic Vue SFC smoke (minimal)

---

Create **two** `.vue` files under `ts-professional-lab/vue-micro/`:

1. `ListPanel.vue`: generic `T` constrained to `{ title: string }`; props `items: T[]`, `renderKey: keyof T` (bonus: enforce `renderKey` extends `keyof T` properly).
2. `ConfirmDialog.vue`: typed emits `confirm`, `cancel` with void vs payload variants.

Ensure `vue-tsc` passes. Note every place you needed `PropType` or generic `defineComponent` escalation.

---

## Lab H — SSR / hydration tabletop (no mega framework required)

---

Write a short **scenario doc**:

- Express-style handler returns HTML with order total from REST.
  - SPA hydrates summary card. List **three plausible mismatch causes** and mitigations (`suppressHydrationWarning` is escape hatch — explain why masking symptoms is dangerous).
- 

## Lab A11y micro-audit

---

Build a rudimentary `DataTable.tsx` **or Vue table** displaying mock rows:

- Wired header ↔ cell association (`scope`, or `thead/tbody` correctness).
  - First focusable filter input on page load; **trap focus** in modal drill (even if mocked). Run automated a11y pass; annotate **two** findings as false positives vs real defects.
- 

## Interview drill bank (self-score)

---

Produce flash answers (≤45s verbal each):

1. Structural vs nominal with example.
2. Generic constraint vs overload when to prefer which.

3. Discriminant property naming pitfalls (kind collisions).
  4. Difference between `unknown`, `any`, and `never` in catch cleanup.
  5. Why `readonly T[]` parameters protect inference better than mutable arrays.
  6. Hydration mismatch triage checklist (5 bullets).
-

## Phase — 04-vue-core-composition-api

---

Directory: 04-vue-core-composition-api/

---

Source: 04-vue-core-composition-api/01-scope-components-reactivity-composition-api.md

## Unit 1 — Scope: Vue 3 core — component, reactivity, boundaries (no router, no Pinia)

**Prerequisite:** Comfortable reading `vue-tsc` diagnostics and keeping `defineProps` / `defineEmits` honest after 03-\*.

Mindset shift: from “template shows data” toward **state owns truth; template is a projection**.

### Learning outcomes

---

- **Component anatomy:** template / script / styles; when a file is too big (ownership split).
- `ref` vs `reactive`: primitives vs object graphs; unwrapping ergonomics in templates.
- **One-way data:** `defineProps` downward; immutability mindset for child inputs.
- **Upward intents:** `defineEmits` — parent retains authority over mutable truth.
- `computed`: derivation without shoving filters/maps into templates; caching semantics.
- `watch`: side effects tied to reactive sources; pairing with debounced search/async (later tied to 06-\* HTTP layer resilience).
- **Lifecycle:** mount vs unmount hooks; fetching on mount and **cleanup** (`onUnmounted`) to prevent leaks.
- **Boundaries:** dashboard → widget decomposition; cohesive ownership slices.
- **Reactivity internals (high level):** Proxy-based dependency tracking → targeted re-render narratives.

Practice spine throughout: `vue-lab/` — **Vue 3 + Composition API only** until routing/Pinia arrives.

---

Source: 04-vue-core-composition-api/02-labs-dashboard-widgets-and-reactivity-drills.md

## Unit 2 — Labs (`vue-lab/` / `dashboard/`)

### Guided builds

---

- **Cards:** `ProductCard`, `UserCard`, `NotificationCard` composing a realistic layout (header/sidebar/content) without dumping 1k-line “god components”.
- **Counters & reactive objects:** derive UI without manual DOM; explain why `count++` updates UI.
- **Props drill:** dashboard → list → leaf; forbid child mutating props; document data flow sentences.
- **Emit flows:** child raises `add-to-cart`; parent commits state mutations.
- **Computed:** totals, filtered lists; contrast with naive methods re-running every render.
- **Watchers:** search typing → debounced side effect (fake fetch placeholder).
- **Lifecycle:** load on mount, cancel/cleanup on unmount; introduce then fix a deliberate listener leak.
- **Reactivity deep-ish:** nested object changes should remain explainable with dependency tracking language.

### Integration (`dashboard/`)

---

Products, cart surface, notifications, stats — document **who owns which truth** and how events cross boundaries.

Interview topics: `ref` vs `reactive`, props vs emit, computed vs watch, lifecycle cleanup, component ownership, reactivity model at whiteboard depth.

---



## Phase — 05-spa-state-vue-router-and-pinia

---

Directory: 05-spa-state-vue-router-and-pinia/

---

Source: 05-spa-state-vue-router-and-pinia/01-scope-router-guards-pinia-and-spa-architecture.md

## Unit 1 — Scope: SPA architecture — router, Pinia, auth, persistence

Mindset shift: from “where do I put this variable” toward **explicit ownership** across routes and stores.

### Learning outcomes

---

- **Router mental model:** URL ↔ matched view; history API without full reload; dynamic params vs static routes.
- **Navigation API:** imperative navigation vs declarative links; understanding back/forward behaviour.
- **Pinia purpose:** kill prop-drilling for cross-cutting client state; multiple focused stores vs one mega-store.
- **Store boundaries:** `cart`, `auth`, `catalog` (examples) with clear responsibilities.
- **Auth arc:** login → token/session → store → guard → protected route; logout semantics.
- **Guards:** `beforeEach` — redirect unauthenticated users; optional role checks (admin vs user).
- **Persistence:** refresh must not erase critical UX state (`cart`); hydration patterns and trade-offs (security later in 09-\*).
- **Async UX per flow:** each remote operation owns **loading / success / error** states (not a global boolean soup).

Practice spine: `shop-spa/` — Vue 3 + **Vue Router** + **Pinia** + fake/Symfony-backed API.

---

Source: 05-spa-state-vue-router-and-pinia/02-labs-shop-spa-routing-auth-and-stores.md

## Unit 2 — Labs (`shop-spa/`)

### Router

---

- Routes for `/`, `/products`, `/products/:id`, `/cart`, `/profile`, optional `/admin`.
- Param extraction and display; programmatic navigation exercises (e.g. after add-to-cart).

### Pinia

---

- Implement **cart**, **auth**, **products** stores with tight APIs.
- Refactor away from prop drilling; document anti-pattern you removed.

### Auth + guards

---

- Fake login; protect `/profile` and `/admin`; redirect with return URL if you want the stretch goal.

### Resilience (preview of 06-\*)

---

- Product list flow with explicit **loading / retry / error** surfaces for at least one API.

### Integration milestone

---

End state: multi-route SPA with **store ownership**, **route ownership**, **guards**, **persistent cart**, documented **why** each boundary exists.

Interview topics: Pinia vs local state, router guards, SPA vs MPA, persistence strategy, navigation flow.

---

## Phase — 06-http-layer-and-resilient-api-integration

---

Directory: 06-http-layer-and-resilient-api-integration/

---

Source: 06-http-layer-and-resilient-api-integration/01-scope-api-ownership-axios-and-failure-modes.md

### Unit 1 — Scope: treating HTTP as a distributed system (frontend view)

Mindset shift: from “I called the endpoint” toward **the UI owns user-visible outcomes of network failure**.

#### Learning outcomes

---

- **Layering**: components call **services/modules**; raw HTTP lives in `api/` (or equivalent), not inside leaf components.
- **HTTP client**: central `ApiClient` with base URL, headers, timeouts, shared interceptors (Symmetry with Symfony API or mocks).
- **Per-request state machine**: idle → loading → success | error | empty; avoid copy-pasted booleans.
- **Error taxonomy**: validation vs network vs server vs auth; map errors to UX (retry, redirect to login, inline message).
- **Retry**: bounded retries with backoff awareness; when *not* to retry (non-idempotent writes without care).
- **Cancellation**: `AbortController` for fast typing / route changes / duplicate inflight calls.
- **Optimistic UI**: instant feedback with explicit rollback when mutation fails.
- **Timeouts & slow APIs**: surface stuck states; contrast with infinite spinners.

Continue extending `shop-spa/` against real or fake backend contracts. Carry **typed seams** from `03-*` (`unknown` ingestion, guards, discriminated API errors) into the HTTP layer implementation—no silent `any` in interceptors.

---

Source: 06-http-layer-and-resilient-api-integration/02-labs-shop-spa-http-hardening.md

### Unit 2 — Labs (`shop-spa/` checkout path)

#### Build tasks

---

- Extract per-domain API modules (`productApi`, `authApi`, `orderApi`, ...) from components.
- Configure **timeouts**, **interceptors** (e.g. attach auth header), and **shared error translation**.
- Implement **loading / empty / error** UX for catalog and cart operations.
- **Retry lab**: inject flaky timeout; observe recovery within limits.
- **Cancellation lab**: product search typing — abort stale requests.
- **Optimistic cart add** with rollback on forced 500.
- **Architecture pass**: document `component` → `service` → `api` → `network` with one sequence diagram or bullet RACI.

#### Deliverable notes

---

Write a short **failure playbook**: what the user sees for 401 / 404 / 500 / timeout / offline-ish behaviour.

Interview checklist: optimistic UI, cancellation, retry policy, timeout ownership, API layer separation, error mapping.

---

## Phase — 07-vue-and-browser-performance

---

Directory: 07-vue-and-browser-performance/

---

Source: 07-vue-and-browser-performance/01-scope-profiling-bundle-split-and-render-cost.md

## Unit 1 — Scope: performance literacy (Vue + browser)

Mindset shift: from “the app loads” toward **measurable cost**: JS evaluation, layout, rendering, memory, repeated network.\*\*

### Learning outcomes

---

- Chrome **Performance**, **Network**, **Memory**, **Vue DevTools** as regular tools — not demos.
- **Bundle economics**: oversized initial JS → code-splitting intuition, tree-shaking relevance, `import()` dynamic chunks.
- **Route-level lazy loading**: defer admin/analytics/heavy screens until needed.
- **Render efficiency**: large lists; avoid accidental  $O(n)$  work per keystroke; `computed` vs eager methods; stable keys.
- **Network**: duplicate inflight calls, chatty waterfalls, payload bloat — pair with 06-\* cancellation/batching habits.
- **Assets**: image weight, lazy images, format trade-offs (high level).
- **Memory**: listener leaks, retained component state, detached DOM — tie back to 02-\*, 04-\* (Vue lifecycle/teardown), and 05-\* (store/router long-lived listeners) as needed.

Ship evidence: before/after metrics (bundle size, main-thread time, request count) for shop-spa/ changes.

---

Source: 07-vue-and-browser-performance/02-labs-shop-spa-performance-rescue.md

## Unit 2 — Labs: performance rescue (shop-spa/)

### Scenario stack (can be staged or combined)

---

- **Large list**: ~1k product rows — profile unnecessary re-renders; fix with derived state and structural discipline.
- **Fat bundle**: introduce then remove bloat — measure pre/post **route-level** `import()`.
- **Lazy routes**: admin / analytics / profile segments load only when visited; verify in Network.
- **Payload / duplicate calls**: reconcile double-fetch patterns; tighten API usage.
- **Images**: optimise product thumbnails; observe transfer size change.
- **Memory**: reproduce listener or store retention leak → fix with teardown.

### Deliverable

---

One-page memo: bottleneck → measurement → mitigation → residual risk/trade-off (e.g. “virtualisation deferred”).

Interview list: lazy loading, code-splitting, profiling, rerender hotspots, Lighthouse limitations, leak diagnosis.

---

## Phase — 08-frontend-testing-vitest-and-e2e

---

Directory: 08-frontend-testing-vitest-and-e2e/

---

Source: 08-frontend-testing-vitest-and-e2e/01-scope-pyramid-contracts-and-regression-safety.md

## Unit 1 — Scope: frontend testing posture (confidence, not coverage theatre)

Mindset shift: from “manual click QA” toward **signals that refactoring is safe enough**.

### Learning outcomes

---

- **Pyramid framing**: pragmatic balance of unit, component/integration, Playwright-class E2E — what each buys.
  - **Unit tests**: pure helpers ( `formatPrice`, `discounts`, `carts`) via **Vitest** (or aligned runner).
  - **Isolation & mocks**: forbid accidental network reach from fast tests; fixture discipline.
  - **Component tests**: **Vue Test Utils** — render, interactions, emits, accessibility smoke where useful.
  - **Store tests**: Pinia actions/mutations boundaries with mocked API seams.
  - **E2E**: thin happy paths + one failure-mode journey (Playwright or equivalent).
  - **Contract drift**: frontend expectations vs backend JSON shape mismatches — how tests catch breakage before prod.
  - **Regression discipline**: purposeful “break tests” exercises to validate signal.
- 

Source: 08-frontend-testing-vitest-and-e2e/02-labs-shop-spa-test-harness.md

## Unit 2 — Labs: build the harness on `shop-spa/`

### Minimum bar

---

- `tests/unit/**` covering money/cart/domain helpers (≥10 purposeful tests).
- Component specs for `ProductCard`, `Cart` surface — props, emits, unhappy UI states.
- Pinia `cartStore` tests with mocked API adapter.
- **Playwright**: login → browse → cart → checkout smoke; second scenario for guarded route.

### Debugging lab

---

Deliver **intentionally broken branches** (wrong field name from API; broken emit wiring) — show failing tests narrowing root cause quickly.

Interview topics: pyramid trade-offs, mocking vs integration, flaky E2E control, regression vs refactor safety.

---

## Phase — 09-production-frontend-deploy-and-security

---

Directory: 09-production-frontend-deploy-and-security/

---

Source: 09-production-frontend-deploy-and-security/01-scope-build-env-deploy-and-observability.md

## Unit 1 — Scope: production frontend (build, deploy, security, ops)

Mindset shift: from “works on localhost” toward **artefacts, environments, cache, and attack surface**.

### Learning outcomes

---

- **Build vs dev**: Vite/ `npm run build` output anatomy; source maps stance; prod minification/size reality.
- **Environment config**: `.env.*` layering; `import.meta.env` (Vite mental model); no secrets baked into client bundles.
- **Delivery path**: artefact → static host / `nginx` container / Compose stack — repeatable local prod-like run.
- **Cache busting**: hashed asset filenames vs stale JS after deploy incidents.
- **Security**: XSS mindset; CSP overview; `httpOnly` **cookies** vs `localStorage` **tokens** — threat-model trade-offs, not zealotry.
- **Security headers** (conceptual CSP, `X-Frame-Options`, HSTS mentions) as infra + app collaboration.
- **Observability**: client error capture hooks (e.g. Sentry-class); bridge user “it broke” reports to breadcrumbs.
- **Incident triage**: cache vs API vs CORS vs timeout hypotheses.

Extend `shop-spa/` deployed beside Symfony/fake backend per your lab constraints.

---

Source: 09-production-frontend-deploy-and-security/02-labs-docker-nginx-incident-drills.md

## Unit 2 — Labs: prod-shaped exercises

### Build & env

---

- Compare dev HMR bundle vs `dist/` output; annotate largest chunks with import graph reasoning.
- Wire **staging vs production** API base URLs purely via env + build artefacts.

### Deploy shape

---

- **Dockerfile + nginx** (or Compose) serving SPA with sensible caching headers vs HTML freshness.

### Incident tabletops

---

- **Stale bundle** after deploy (users on old hashed JS): detection + remediation narrative.
- **Checkout timeout** separating browser vs CDN vs API path.
- **CORS regression** vs misconfigured env URLs.
- **Global error spike** traced via monitoring breadcrumbs.

### Checklists

---

Maintain a **release checklist**: build validation, smoke, monitoring deltas, rollback path.

Interview focus: XSS/CSP/token storage trade-offs, hashed assets + cache busting, header roles, rollback, env separation.

---

## Phase — 10-interview-drills-system-design-whiteboard

---

Directory: 10-interview-drills-system-design-whiteboard/

---

Source: 10-interview-drills-system-design-whiteboard/01-scope-debugging-tradeoffs-and-enterprise-scenarios.md

## Unit 1 — Scope: senior interview rehearsal (JS + TypeScript + Vue + SPA systems)

Mindset shift: from “knowing definitions” toward **articulating hypotheses, trade-offs, fixes, and documentation.**

### Working pattern (reuse for every drill)

---

problem → observable symptom → investigation plan → hypotheses → validation steps → proposed fix → trade-offs → documented outcome.

### Module map

---

1. **JS runtime traps:** synthetic event-loop/ `Promise / setTimeout` orderings + closure memory stories.
2. **TypeScript depth:** generics + constraints, discriminated unions + exhaustiveness, guards vs schema validation, `satisfies / as const`, branded IDs, module/ `import type` hygiene, composition vs inheritance for DTO layers.
3. **Vue internals:** reactive graph, intentional vs accidental rerenders, watcher misuse, `vue-tsc` failure triage.
4. **SPA/system design:** whiteboard ecommerce modules (catalog, cart, checkout, notifications) with router + store boundaries.
5. **API adversity:** stalled checkout, flaky retries, cancellation, optimistic rollback storytelling.
6. **Production pathology:** stale assets post-deploy, CORS regressions, memory growth, unexplained timeouts.
7. **Performance narratives:** oversized bundles, gigantic lists — tie actions to tooling readouts.

Use prior phase artefacts ( `frontend-lab`, `browser-lab`, `ts-professional-lab`, `vue-lab`, `** shop-spa/**` ) instead of toy isolated snippets wherever possible.

---

Source: 10-interview-drills-system-design-whiteboard/02-labs-eight-week-pressure-scenarios.md

## Unit 2 — Labs: scripted pressure scenarios

### Week-style rotation (adapt pacing freely)

---

1. **Event-loop worksheet:** annotate console order for layered `microtask`/timer puzzles; verbalise closures holding live memory across scopes.
2. **Type-level incident:** widen a union accidentally in a mapper; restore exhaustiveness; explain why `any` hid a production bug.
3. **Reactivity/debug:** sluggish dashboard — hypothesis list + Vue DevTools plan before editing code.
4. **SPA architecture whiteboard:** draw ownership slices for ecommerce routes/stores/services.
5. **API failure tabletop:** timeouts + rollback of optimistic UX with user-safe messaging plan.
6. **Production tabletop:** stale JS + phantom API success + UI timeout — differentiate layers.
7. **Performance tabletop:** huge dataset + sluggish bundle — backlog of mitigations prioritised by risk/effort.
8. **Enterprise capstone (~90 min silent):** Symphony API + SPA + cascading symptoms — document investigation trail only (no spoilers in chat transcripts).
9. **Final scavenger hunt on** `shop-spa/` : inject intersecting defects (auth, retry, stale cache, rerenders, leaks) → single RC write-up capturing root cause vs symptom.

Interview checklist echoes original roadmap breadth: closures & event loop → **TypeScript narrowing & contracts** → Vue through deploy/CSP/cache/monitoring narratives.

---